PROJECT IMPLEMENTATION ROADMAP

Design and Development of an Explainability-Driven Hallucination Detection Framework for Offline Retrieval-Augmented Generation

Course / Project:	S2 M.Tech Main Project
Domain Area:	Natural Language Processing (NLP) & Explainable AI (XAI)
Implementation:	Fully Offline Deployment (No API Dependencies)
Key Focus:	Intrinsic Hallucination Detection & Citation-Grounding Logic

Executive Summary

Large Language Models (LLMs) deployed within Retrieval-Augmented Generation (RAG) pipelines are highly susceptible to *hallucinations*—generating claims that are unsupported or contradicted by retrieved context. In offline, high-security, or resource-constrained environments, standard cloud-based fact-checking models and commercial APIs are completely unusable. This project establishes an **entirely offline, explainability-driven framework** that leverages internal retrieval signals (similarity heatmaps, document-chunk attribution, provenance, and confidence metrics) as primary features for real-time hallucination detection and response filtering.

- **Intrinsically Grounded:** Leverages internal metrics generated directly during retrieval and inference rather than relying on heavy post-hoc external validation models.
- **Robust Fallbacks:** Employs a decision controller to either output verified claims with citations, return a partial response warning the user of weak grounding, or block the output entirely to prevent misinformation.
- **Privacy & Speed:** Designed to run locally on consumer-grade workstations using optimized local models (sentence-transformers, quantized LLMs, and lightweight NLI cross-encoders).

Problem Formulation & Core Objectives

The Problem Statement

Despite RAG providing grounding context, generative LLMs frequently output claims that are weakly supported or contradicted by retrieved evidence. Traditional hallucination mitigation relies on heavy, black-box external services or cloud APIs (e.g. OpenAI GPT-4 API) for validation. These methods introduce high latency, compromise data privacy, and are completely infeasible for offline installations. Meanwhile, internal explainability signals—such as cosine similarity scores, chunk attribution vectors, and base token probabilities—remain vastly underutilized as features for real-time hallucination control.

CORE RESEARCH QUESTION: The core research question of this M.Tech project is: *How can internal explainability signals generated within an offline RAG framework be systematically combined with lightweight local NLP models to detect, isolate, and suppress hallucinated outputs, while ensuring transparency and evidence grounding?*

Project Objectives

- **Design Local Ingestion & Retrieval Core:** Build a local knowledge base using document parsers, token-level chunking, and FAISS indexing for fast vector lookup.
- **Formulate an Explainability Layer:** Capture cosine similarity, chunk-source mappings, and model-level confidence outputs to trace each generated statement to its exact source chunk.
- **Develop Multi-Technique Hallucination Detection:** Combine lightweight local Natural Language Inference (NLI) entailment, token-level BERTScore, and vector similarity thresholds to detect logical and semantic hallucination types.
- **Implement a Tri-State Decision Controller:** Execute rule-based response filtering to output answers (with precise inline citations), deliver partial/warned answers, or completely block untrusted responses.
- **Deploy an Interactive Web Demo:** Construct a local Streamlit interface that visualizes retrieved evidence, citation linkages, NLI classification outputs, and attention maps.

Core Novelty: Intrinsic Explainability Co-integration

Most RAG guardrails perform *post-hoc checking*—treating the system as a black box and passing the final text to a secondary LLM for verification. This is computationally expensive and slow. **This project's novelty lies in co-integrating explainability and detection:** using the scores calculated during document retrieval and generation as primary inputs to the detector, creating an intrinsic, lightweight feedback loop that operates natively within the pipeline.

Proposed System Architecture

The system is structured as an end-to-end local pipeline where explainability signals and hallucination detection layers are deeply interwoven. Rather than executing as isolated blocks, the retrieval, generation, and validation phases exchange metrics to control the final output.

Pipeline Flow Sequence

- **1. Query Ingestion & Embedding:** The user query is tokenized and converted to a dense vector using the local *all-MiniLM-L6-v2* embedding model.
- **2. Context Retrieval:** FAISS performs a fast cosine similarity search against the document store and retrieves the top-*K* chunks. Similarity scores are saved.
- **3. Local LLM Generation:** The query and retrieved context are formatted into a system prompt and passed to the local LLM (e.g. *Mistral-7B-Instruct* via Ollama). The model generates the raw response.
- **4. Claim Segmentation:** The generated response is split into individual sentences (claims) for granular inspection.
- **5. Triple-Verification Detection:** Each claim is verified against retrieved chunks using: (a) NLI Entailment scoring (Cross-Encoder), (b) Token-level semantic overlap (BERTScore), and (c) Vector similarity.
- **6. Metric Aggregation:** The controller calculates a *Provenance/Support Score* for each claim, incorporating retrieval cosine scores and NLI probabilities.
- **7. Tri-State Control Action:** The controller selects the final output pathway: *Answer with Citations*, *Partial Answer with Warned Claims*, or *Block & Clarify*.

System Architecture Visual Flow Map

```
[ User Query ] ------> [ all-MiniLM-L6-v2 Embedder ] -------> [ FAISS Vector DB ] | (Top-K Chunks + Sim Scores) | [ Output Response ] <--- [ Decision Controller ] <--- [ Explainability Layer ] ^ | (Tri-State) (Attribution, Sim, NLI, BERTScore) | v [ Block / Partial / Cite ] <---- [ Hallucination Detector ]
```

Detailed Technology Stack Selection

All selected technologies are open-source, lightweight, and capable of running fully offline on consumer-grade desktop hardware. This ensures data privacy and zero operating costs.

Layer-by-Layer Components

Pipeline Layer	Selected Tool / Model	Technical Justification
Document Embeddings	all-MiniLM-L6-v2	Extremely compact (80MB), fast CPU inference, 384-dimensional dense vectors optimized for semantic text similarity.
Vector database	FAISS (Facebook AI Search)	Runs entirely in-memory, serverless, compiled C++ backend, enabling sub-millisecond retrieval without network overhead.
Local LLM Generator	Mistral-7B-Instruct / LLaMA-3-8B (Q4_K_M GGUF)	High-performance open-source models, quantized to 4-bit to run efficiently within 8GB VRAM using Ollama/llama.cpp.
NLI Entailment Classifier	cross-encoder/nli-deberta-v3-large	Fine-tuned DeBERTa model that excels at sentence-level implication (Entails/Neutral/Contradicts) with higher precision than raw LLMs.
Semantic Overlap	BERTScore (Local)	Token-level similarity using contextual embeddings; avoids lexical match limits without calling cloud services.
Orchestration & UI	Python, LangChain, Streamlit	Streamlit provides rapid dashboard prototyping, while LangChain manages local model inputs/outputs and chunk parsing.

Alternative Technologies Evaluated & Rejected

Component	Rejected Alternative	Reason for Rejection
Embeddings	OpenAI text-embedding-ada-002	Requires cloud connection, violates 'offline' mandate, incurs recurring API usage costs.
Vector DB	Pinecone / Milvus Server	Pinecone is cloud-only. Milvus requires dedicated docker services, causing high overhead for desktop deployment.
Generator LLM	GPT-4 / ChatGPT API	Black-box nature blocks access to internal model probabilities, attention weights, and requires active internet.
Detector	Heavyweight LLM Prompting	Running a 70B parameter model just to evaluate the output of a 7B model is computationally redundant and introduces extreme latency.

Dataset and Evaluation Strategy

To rigorously validate the hallucination detection framework, the evaluation methodology employs three distinct data streams: a primary domain-specific knowledge corpus, established public evaluation benchmarks, and a custom-labeled gold standard dataset.

1. Primary Domain Knowledge Corpus

The system will be tested on a specialized technical domain to demonstrate adaptation. The chosen domain is **Technical Documentation / Software Engineering Manuals** (specifically, Git Version Control Documentation, Docker reference manuals, and Python developer guides). This domain provides structured, fact-dense text where hallucinated arguments, command flags, or syntax would be instantly fatal.

2. Hallucination & Truthfulness Benchmarks

To evaluate the detector's classification accuracy independent of the specific domain corpus, the system will run against samples from the following public datasets:

- **RAGTruth:** A fine-grained hallucination corpus consisting of 17,790 LLM responses across QA, summarization, and data-to-text tasks, complete with word-level and sentence-level hallucination labels.
- **HaluEval:** A large benchmark containing over 30,000 generated responses with paired factual and hallucinated texts, specifically designed for testing detection models.

3. Custom Golden Evaluation Dataset Setup

To evaluate the end-to-end system under typical M.Tech project conditions, a custom test set of **150 QA pairs** will be curated and labeled:

- **Dataset split:** 50 questions with complete evidence support in the corpus, 50 questions with partial evidence support, and 50 questions designed to trigger out-of-corpus hallucinations (queries about topics not contained in the documents).
- **Ground Truth Annotation:** Each generated claim will be manually annotated as: (1) *Fully Supported*, (2) *Partially Supported*, or (3) *Hallucinated / Unsupported*.

METRIC THRESHOLD TUNING: This custom evaluation set will serve as the validation ground for establishing the optimal threshold values for the NLI, BERTScore, and Cosine similarity metrics in the decision controller.

Evaluation Framework Overview Table

Dataset Type	Source / Model	Purpose	Target Metric
Knowledge Corpus	Local Tech Docs (PDFs/MDs)	Supplies context chunks to LLM	Retrieval Recall@K, MRR
Public Benchmark	RAGTruth / HaluEval	Tests detector's generalizability	AUC-ROC, F1-Score
Custom Golden Set	150 Hand-curated QA Pairs	Tunes decision thresholds	Precision / Recall trade-off

Project Roadmap & Implementation Phases

The implementation plan spans 16 weeks, covering ingestion core creation, explainability tracking, metric formulation, threshold optimization, web demo development, and thesis compilation.

Phase & Weeks	Core Tasks & Implementation Details	Key Deliverables / Milestone
Phase 1 (W1 - W2)	Set up local virtual environment. Implement PDF/Markdown text extraction using python-docx and PyPDF2. Write token-based sliding-window chunker.	Data ingestion script that processes folders and outputs text chunks.
Phase 2 (W3 - W4)	Integrate all-MiniLM-L6-v2. Create FAISS index script. Optimize retrieval parameters (chunk size, overlap, K retrieved chunks).	Retrieval module yielding top-K chunks with similarity scores.
Phase 3 (W5 - W6)	Configure local Mistral 7B LLM via Ollama. Build generator prompt wrapper. Write chunk-to-sentence attribution tracer.	Local generator script returning raw answers mapped to source chunks.
Phase 4 (W7 - W9)	Implement NLI classifier (cross-encoder). Implement local BERTScore. Formulate cosine threshold filters.	Detection engine returning NLI, BERTScore, and cosine stats per sentence.
Phase 5 (W10 - W11)	Design aggregation logic for the Provenance Score. Implement Tri-State controller (citations, warnings, blocking).	Decision engine that filters answers based on support levels.
Phase 6 (W12 - W14)	Develop local Streamlit web application. Run benchmarks on RAGTruth/HaluEval. Plot ROC/Precision-Recall curves.	Streamlit UI demo and comprehensive performance evaluation charts.
Phase 7 (W15 - W16)	Perform error analysis. Compile thesis chapters. Conduct review defense preparation.	Final M.Tech Project Report and slide deck ready for defense.

Critical Path & Milestones Graph

```
W1-2: Chunks ===> W3-4: FAISS ===> W5-6: Local LLM ===> W7-9: Detector || W15-16: Thesis <===
W12-14: Demo UI <=== W10-11: Controller <====//
```

System Evaluation & Demo Visualization Plan

1. Evaluation Metrics

Metric Category	Specific Metrics Used	Formula / Concept	Target Value
Retrieval	Recall@K, MRR (Mean Reciprocal Rank)	Percentage of relevant chunks in top-K; position of first relevant chunk.	Recall@3 >= 0.85 MRR >= 0.80
Detection Accuracy	F1-Score, AUC-ROC, Precision	Harmonic mean of precision and recall. Trade-off curve for hallucination vs truth.	F1 >= 0.82 AUC-ROC >= 0.88
Faithfulness	Provenance/Support Score, Citation Coverage	Average NLI entailment probability across all sentences. Citation density.	Support Score >= 0.85 Coverage = 100%
System Performance	Inference Latency, Memory Footprint	Time in seconds to generate and verify answer. GPU/RAM consumption.	Latency <= 5.0s Memory <= 12GB RAM

2. Web Demo Visualization Layout (Streamlit Dashboard)

The Streamlit dashboard serves as the user-facing demonstration and visual proof of the system's explainability features. The layout is divided into three primary segments:

- **Query & Generation Box:** Input text field for user query. Real-time answer generation box showing words as they stream, and color-coding sentences based on support (Green = Entailed, Orange = Weak Support, Red = Contradicted/Unsupported).
- **Explainability Panel:** Toggleable view displaying attention heatmaps over retrieved chunks, cosine similarity bar charts for retrieved documents, and direct mapping indicating which sentence came from which source chunk.
- **Safety Gauges & Logs:** Speedometer showing the aggregated Provenance Score (0 to 100%). Log console printing model decisions: *'ACTION: Cite source'* or *'ACTION: Suppressed claim #3 due to NLI Contradiction'*.

LOCAL DEPLOYMENT CONFIG: The Streamlit UI runs entirely on localhost:8501, loading the models locally via Hugging Face pipelines and Ollama endpoints, preserving the strictly offline nature of the framework.

M.Tech Thesis Structure & Academic Outline

The project report/thesis will be organized into six academic chapters in accordance with standard graduate engineering guidelines, focusing on mathematical formulation, experimental rigor, and system validation.

Chapter 1: Introduction

Background on Generative AI and RAG; The threat of factual hallucinations; Contextualizing the problem in offline environments; Research questions, scope, and objectives of the work; Document structure overview.

Chapter 2: Literature Review

Historical overview of RAG systems; Survey of hallucination types (intrinsic vs. extrinsic); Critical review of post-hoc fact checking vs. internal explainability; Entailment, BERTScore, and semantic metrics; Identification of current research gaps.

Chapter 3: System Design and Architecture

Detailed pipeline architecture; Mathematical modeling of explainability indicators; Formula for combining NLI entailment probabilities, BERTScore weights, and cosine similarity; State transitions of the Decision Controller.

Chapter 4: Implementation Methodology

Setting up the offline pipeline; Local models configuration (all-MiniLM-L6-v2, Cross-Encoder, quantized Mistral-7B); Tokenization and vector store construction with FAISS; Custom paragraph and claim parser development.

Chapter 5: Experimental Results and Analysis

Testing corpus details; Verification of retrieval accuracy (MRR, Recall@K); Precision-Recall performance of the NLI detector; ROC curves and optimal decision threshold calibration; Latency vs. accuracy trade-off curves.

Chapter 6: Conclusion and Future Work

Summary of contributions; Answers to primary research questions; Project limitations (hardware constraints, vocabulary overlap); Suggestions for future extensions (attention map extraction, multi-lingual adaptation).

REPORT FORMATTING RULES: All bibliography entries will follow the standard IEEE formatting guidelines, ensuring high-quality citations of major conferences (ACL, EMNLP, NeurIPS) and journals (IEEE Access, IEEE TKDE).